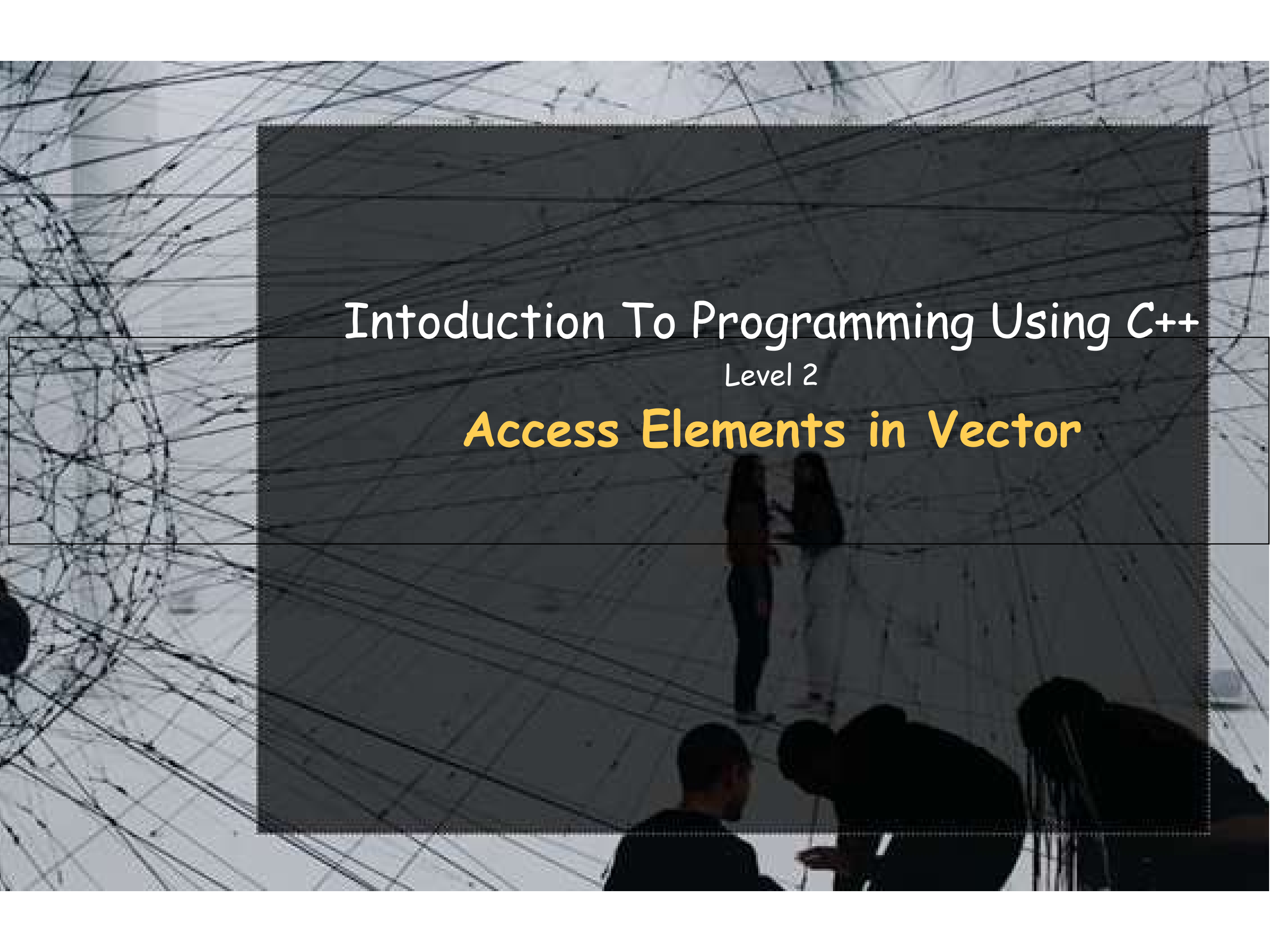


# Introduction To Programming Using C++ Level 2

## Access Elements in Vector



# Introduction To Programming Using C++

Level 2

## Access Elements in Vector

# Definition

## Accessing Vector Elements

 In C++, there are two main ways to access elements in a `vector` :

1  `.at(index)`

2  `[index]`

# Explanation

## ❑ Using `.at(index)`

- ▣ `.at(index)` is used to access the element at a specific position inside the `vector`.

- ▣ It checks whether the index is within the valid range.

- ▣ If the index is out of range, it throws an `out_of_range` exception.

## ❑ Using `[index]`

- ▣ The `[]` operator provides direct access to an element at a specific index.

- ▣ It does not perform bounds checking.

- ▣ This makes it slightly faster because no range check is performed.

- ▣ If the index is outside the valid range, using `[]` results in undefined behavior.

# 💡 Example

```
int main()
{
    vector<int> num{1, 2, 3, 4, 5};
    cout << "\nUsing .at(i):\n";
    cout << "Element at Index 0: " << num.at(0) << endl;
    cout << "Element at Index 2: " << num.at(2) << endl;
    cout << "Element at Index 4: " << num.at(4) << endl;
    cout << "\nUsing [i]:\n";
    cout << "Element at Index 0: " << num[0] << endl;
    cout << "Element at Index 2: " << num[2] << endl;
    cout << "Element at Index 4: " << num[4] << endl;
    return 0;
}
```

## ➡ Result

Using .at(i):

Element at Index 0: 1

Element at Index 2: 3

Element at Index 4: 5

Using [i]:

Element at Index 0: 1

Element at Index 2: 3

Element at Index 4: 5

# Comparison

## `.at(index)`

- ✓ Checks whether the index is within range.
- ✓ Throws an `out_of_range` exception if the index is invalid.
- ✓ Slightly more overhead because it performs a bounds check.
- ✓ Useful when you want bounds checking.

## `[index]`

- ✓ Does not check whether the index is within range.
- ✓ Provides direct access to the element.
- ✓ Slightly less overhead.
- ✓ Using an invalid index results in undefined behavior.
- ✓ Useful when you know the index is valid.

## ✦✦ Summary

- ✓ Use `.at()` when you want bounds checking and safer access.
- ✓ Use `[]` when you are sure that the index is valid and want direct access.
- ✓ `.at()` throws an `out_of_range` exception when the index is invalid.
- ✓ `[]` does not check the index, and an invalid index results in undefined behavior.

Thank you 😊